

Rapport de Synthèse : Audit et Architecture de Détection

Projet : SAE 5 Cyber & DevCloud - "Nixie" | Maxine Botturi & Sirine Ben Kadhi

Date : Janvier 2026

Destinataires : Direction Technique / Client

Répertoire du projet : https://github.com/Radiant-Maxine/SAE_5C03_Nixie/

1. Résumé Exécutif

Dans le cadre de la sécurisation d'un environnement Active Directory sensible, notre équipe a déployé et comparé plusieurs solutions de surveillance et de détection (basé sur l'infrastructure *Game of Active Directory*). Ce "Proof of Concept" (PoC) visait à déterminer la stack défensive la plus efficace, la plus simple à déployer et offrant la meilleure visibilité sur les menaces.

Notre conclusion est sans appel : bien que la stack Elastic offre une puissance d'analyse brute considérable, **la solution Wazuh, couplée à l'IDS Suricata, représente le meilleur compromis efficacité/maintenabilité pour votre environnement.**

2. Architecture Mise en Œuvre

Pour garantir une objectivité totale, nous avons déployé deux chaînes de détection parallèles sur l'infrastructure ESSOS.LOCAL et SEVENTHKINGDOM.LOCAL :

- **Collecte de Logs Centralisée (OpenWEC) :**
 - Mise en place de **deux serveurs OpenWEC conteneurisés** (Docker), isolant la collecte de logs par domaine pour plus de sécurité et de performance.
 - Déploiement de **Sysmon** et configuration de l'audit Windows via GPO pour une remontée automatique et granulaire des événements.
- **Analyse et Détection (SIEM & IDS) :**
 - **Stack A :** Serveur **Elastic** (ELK) avec agents déployés via Ansible.
 - **Stack B :** Serveur **Wazuh** avec agents déployés via Ansible.
 - **Surveillance Réseau :** Sonde IDS **Suricata** positionnée en amont pour l'analyse des flux réseaux.
- **Innovation & Visualisation :**
 - Développement d'outils Python intégrant des **fonctionnalités LLM** (IA) pour l'analyse sémantique des logs OpenWEC.
 - Création d'un **Dashboard Streamlit** pour une visualisation synthétique des métriques de sécurité.

3. Validation Offensive : Scénarios d'Attaque

Pour éprouver la robustesse de ces solutions en conditions réelles, nous avons exécuté deux campagnes d'attaques distinctes sur le domaine ESSOS.LOCAL. Ces scénarios nous ont permis de mesurer la réactivité de chaque outil :

1. **Scénario 1 (Faible Intensité) :** Simulation d'une phase de reconnaissance (scan de ports) et tentatives d'accès initiales (brute-force SMB). *Objectif : Tester les seuils de détection de base.*

2. **Scénario 2 (Haute Intensité)** : Simulation d'une compromission avancée incluant l'exploitation de vulnérabilités (ZeroLogon), et la connexion distante (SMB). *Objectif : Valider la capacité de corrélation d'événements critiques.*

4. Comparatif des Solutions (SIEM)

Suite à l'analyse des logs générés par ces attaques, voici notre comparatif :

Critère	Wazuh (Recommandé)	Elastic (ELK)
Déploiement	Rapide : L'automatisation Ansible a permis un déploiement des agents quasi-instantané et sans friction.	Complexe : La configuration nécessite plus de finesse et de ressources serveur.
Détection	Native : Les règles par défaut (SCA, Rootcheck) et la corrélation d'événements sont immédiatement opérationnelles.	À construire : Demande un effort important de création de règles et de tuning pour atteindre le même niveau d'alerte.
Lisibilité	Excellente : Les alertes sont claires, catégorisées et directement exploitables pour la réponse à incident.	Densité : Très puissant pour la recherche (Threat Hunting) mais peut noyer l'analyste sous le volume de données brutes.
Réponse	Intègre nativement l' Active Response pour bloquer les menaces automatiquement.	Nécessite des intégrations tierces pour l'automatisation de la réponse.

Verdict : Wazuh s'impose par sa capacité à fournir de la valeur immédiatement ("Out of the box") tout en restant flexible.

5. La Nécessité de l'IDS Suricata

La question "Dois-je mettre un système IDS en place ?" reçoit une réponse positive ferme.

1. **Complémentarité** : Alors que Sysmon/Wazuh voient ce qui se passe *sur* la machine (processus, fichiers), Suricata voit ce qui transite *entre* les machines.
2. **Preuve par l'exemple** : Lors de nos tests d'intrusion (notamment sur les mouvements latéraux), Suricata a levé des alertes sur des signatures de trafic malveillant que les logs systèmes seuls n'auraient pas permis de qualifier avec certitude.
3. **Visibilité** : Il est le seul rempart capable de détecter des tentatives d'exploitation réseau avant même qu'elles n'atteignent leur cible.

6. Recommandations

1. **Adopter le couple Wazuh + Suricata** comme socle de votre SOC (Security Operations Center).
2. **Maintenir l'architecture OpenWEC** : Elle découple la collecte de l'analyse, permettant de changer de SIEM à l'avenir sans reconfigurer tous les postes clients.
3. **Industrialiser les Scripts IA** : Nos prototypes de traitement de logs par LLM ont montré un potentiel fort pour réduire la fatigue des analystes en pré-qualifiant les incidents.

Rapport Technique et Stratégique de Déploiement d'une Infrastructure de Défense Cybernétique (Blue Team) en Environnement Hostile (GOAD) : Projet SAE 5C03 "Nixie"



Résumé Exécutif

Ce document constitue le rapport final détaillé du projet de la SAE 5C03 du groupe "Nixie", réalisé dans le cadre du Bachelor Universitaire de Technologie (B.U.T.) en Réseaux et Télécommunications, parcours Cybersécurité. L'objectif principal de ce projet était de concevoir, déployer et valider une infrastructure de sécurité défensive (Blue Team) complète, capable de surveiller un système d'information d'entreprise simulé et vulnérable, basé sur le framework *Game of Active Directory* (GOAD) et de valider des attaques sur ces serveurs vulnérables (Red Team).

Face à la sophistication croissante des menaces persistantes avancées (APT) et à la complexité des environnements Active Directory, nous avons au cours du projet adopté une approche de "Défense en Profondeur". Nous avons orchestré le déploiement automatisé d'une stack de détection hétérogène comprenant **OpenWEC** pour la centralisation des journaux Windows, **Elastic Stack** (Kibana, Elasticsearch, Fleet...) pour l'analyse de données massives, **Wazuh** pour la corrélation de sécurité et la réponse active, ainsi que **Suricata** pour la surveillance des flux réseaux. L'innovation majeure de notre projet par rapport aux autres réside dans l'intégration de mécanismes d'intelligence artificielle (LLM) via des tableaux de bord **Streamlit** pour synthétiser l'analyse de logs, et l'utilisation intensive de l'automatisation via **Ansible** et les objets de stratégie de groupe (GPO).

L'analyse comparative issue de nos tests d'intrusion (Red Teaming) menés depuis l'environnement **Exegol** démontre que si la pile Elastic offre une flexibilité inégalée pour l'exploration de données, **Wazuh** s'impose comme la solution SIEM la plus opérationnelle pour une détection immédiate des menaces Active Directory critiques telles que le *Kerberoasting* ou le *DCSync*. De plus, le déploiement d'**OpenWEC** sous forme conteneurisée valide une architecture de collecte de journaux résiliente et économiquement viable, s'affranchissant des coûts de licence Windows Server pour les collecteurs.

0. Gestion de Projet : Cadre Méthodologique et Agile

La réussite d'un projet d'ingénierie complexe, impliquant l'interconnexion de systèmes hétérogènes (Linux, Windows, Docker) et de technologies de pointe (SIEM, IDS, LLM), repose sur une gestion rigoureuse des ressources et du temps. Nous avons adopté une méthodologie **Agile**, adaptée aux contraintes d'une équipe restreinte de deux spécialistes en cybersécurité opérant sur un cycle court et intense.

0.1. Application de la Méthodologie Agile

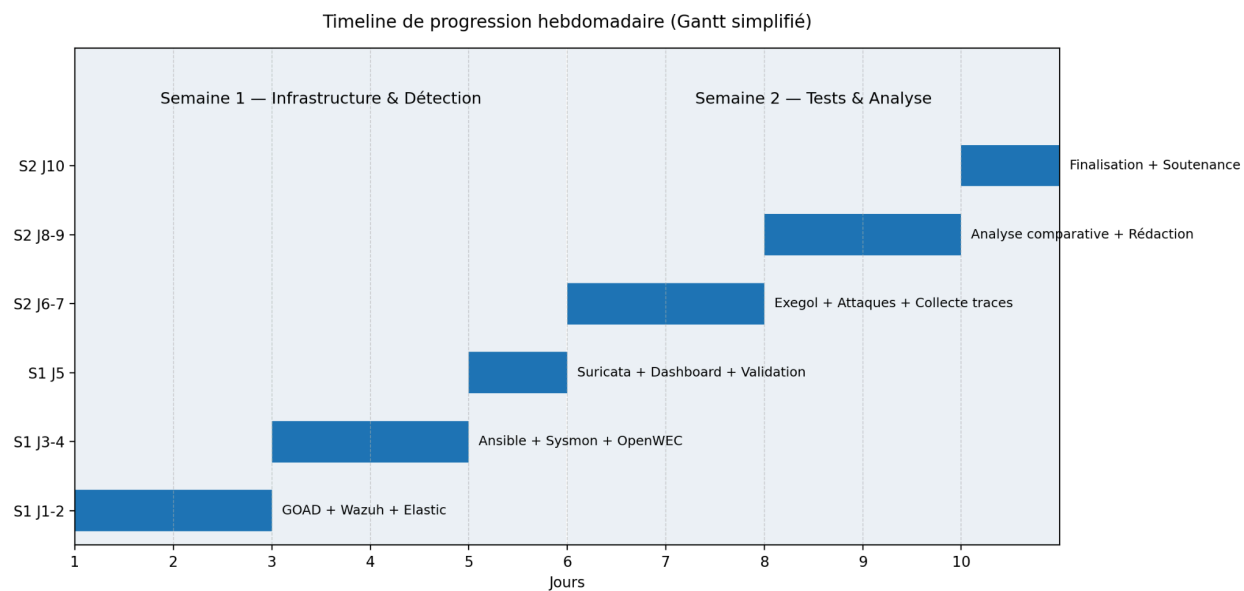
Contrairement aux approches traditionnelles en cascade (Waterfall), qui rigidifient les phases de conception et de déploiement, l'approche Agile nous a permis d'itérer rapidement sur l'architecture de détection. Le projet a été segmenté en "sprints" fonctionnels, permettant de livrer des briques technologiques opérationnelles à intervalles réguliers.

Cette flexibilité s'est avérée cruciale lors de l'intégration d'**OpenWEC**. Initialement, nous avions envisagé un déploiement classique sur machine virtuelle. Cependant, face aux contraintes de ressources du laboratoire virtuel, nous avons pivoté vers une architecture conteneurisée via Docker lors du deuxième sprint, une adaptation rendue possible par notre cycle de décision court.

Quantification de l'Effort et Répartition des Tâches

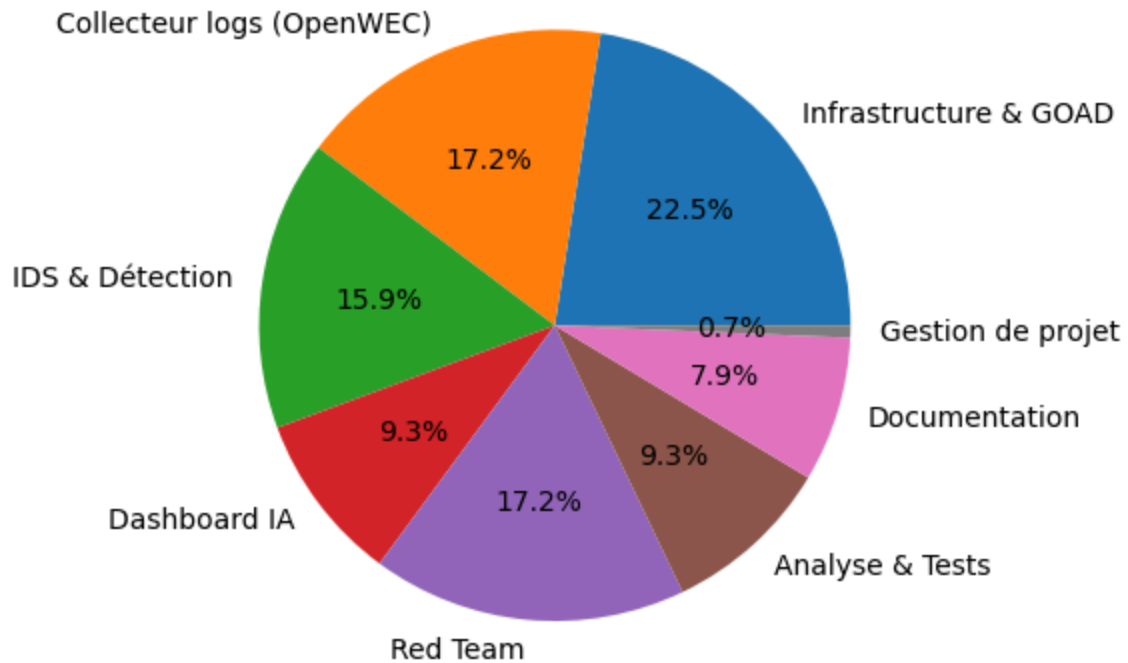
Au final, nous avons pu profiter de notre utilisation d'un Diagramme de Gantt pour pouvoir gérer par nous mêmes les échelons et l'organisation des tâches. Au final, nous sommes

arrivés sur cette timeline simplifiée :



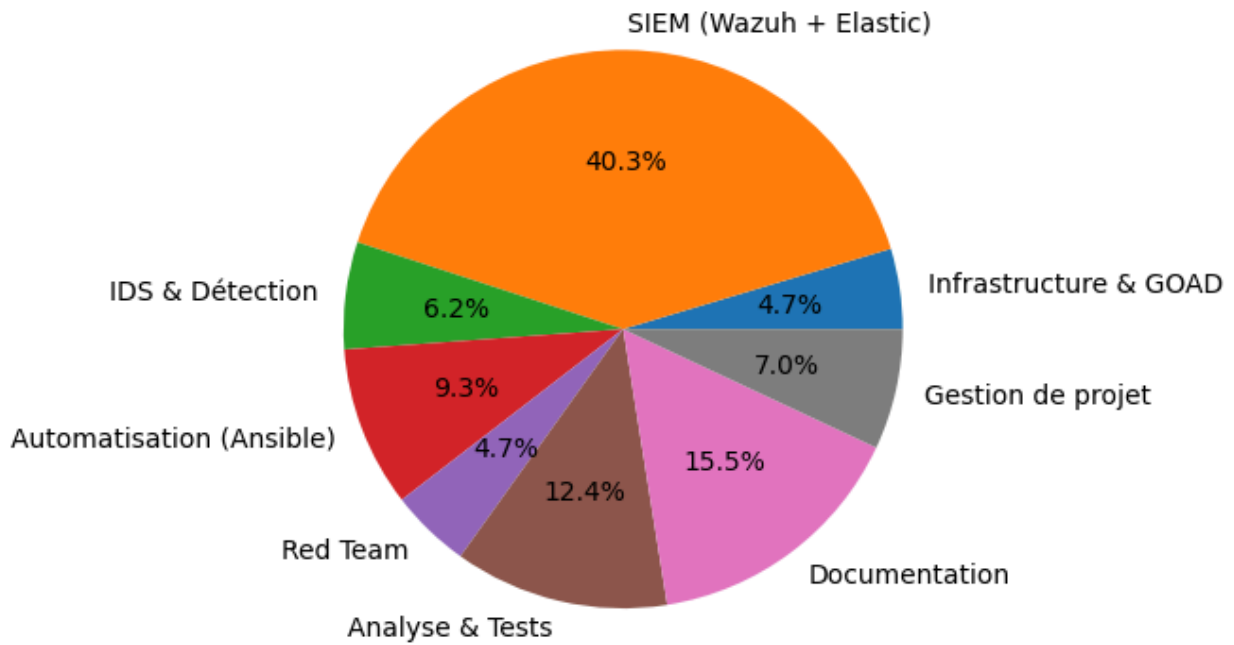
Pour regarder plus dans les détails, nous avons fais une répartition Temps/Tâches/Personne afin de voir qui a fais quoi et quelles ont été les tâches les plus chronophages. Ainsi, pour la première membre de l'équipe cette répartition ressemble à ça :

Répartition des tâches – Maxine (sans tâches à 0%)

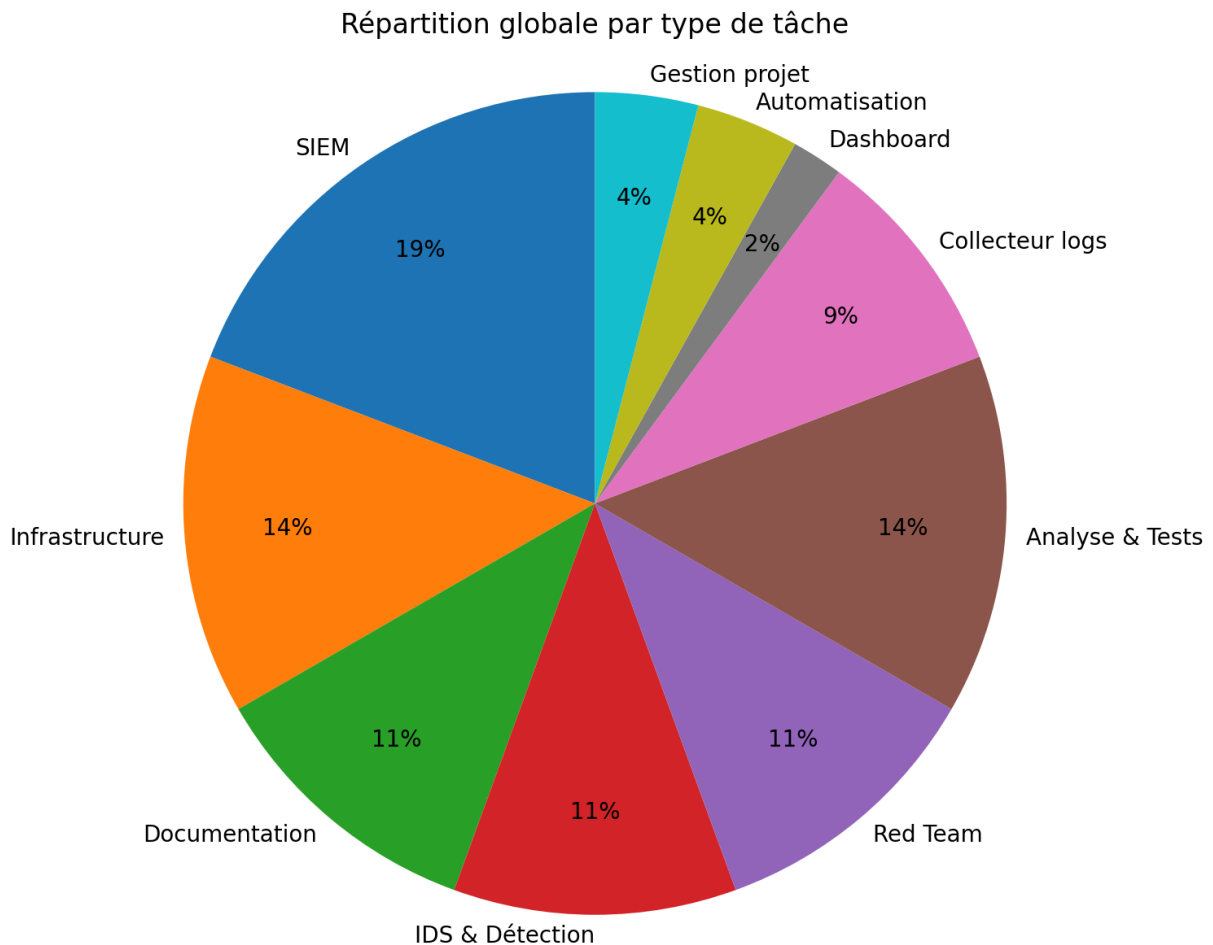


Et pour la seconde membre de l'équipe, elle ressemble à ça :

Répartition des tâches – Syrine (sans tâches à 0%)



Ainsi au total nous avons exactement cette répartition de tâches dans le projet :



0.2. Intégration de l'Intelligence Artificielle Générative (GenAI)

L'une des particularités de ce projet réside dans l'utilisation proactive des intelligences artificielles **Gemini** et **Perplexity** comme accélérateurs de développement et assistants opérationnels.

- **Gestion du Versioning (Git) :** L'IA a été utilisée pour générer des messages de commit conventionnels et structurés, garantissant une lisibilité optimale de l'historique du projet. Elle a également assisté la gestion du répertoire GitHub, réduisant les conflits de fusion lors du travail collaboratif sur les playbooks Ansible.
- **Documentation Technique :** La rédaction des documentations (README, guides d'installation) a été assistée par IA pour synthétiser les procédures techniques complexes en instructions claires et reproductibles. Cela assure la pérennité du projet et facilite la reprise par d'autres équipes, tout en permettant à notre équipe d'accélérer nos process pour mettre en place la meilleure infrastructure et les meilleures analyses.
- **Débogage et Scripting :** Face à des erreurs cryptiques lors du déploiement des conteneurs OpenWEC (problèmes de certificats SSL/TLS ou de configuration GPO),

Perplexity a permis d'identifier rapidement les solutions en corrélant les documentations officielles et les discussions communautaires (GitHub Issues, Stack Overflow).

Cette synergie Humain-IA a permis de maintenir un haut niveau de qualité technique malgré les délais restreints, illustrant l'évolution du métier d'ingénieur cybersécurité vers un rôle de superviseur d'outils intelligents.

1. Architecture Mise en Place et Configuration Réseau

L'architecture du projet a été conçue pour simuler fidèlement un réseau d'entreprise vulnérable tout en isolant les composants de sécurité et d'attaque. Cette ségrégation est fondamentale pour empêcher qu'une compromission du domaine cible ne permette à l'attaquant de désactiver ou de manipuler les outils de surveillance.

1.1. Le Cœur de la Cible : Environnement GOAD (ESSOS.LOCAL)

Nous avons déployé le projet **GOAD (Game of Active Directory)**, développé par Orange Cyberdefense, pour servir de terrain de jeu réaliste.¹ Contrairement aux environnements de laboratoire simplistes, GOAD déploie une forêt Active Directory complète avec des relations d'approbation et des vulnérabilités configurationnelles intrinsèques (et non pas seulement des CVEs non patchées).

Notre périmètre s'est concentré sur le domaine **ESSOS.LOCAL**, qui représente une filiale ou une entité distincte au sein de l'univers GOAD. Ce choix est stratégique : il permet d'étudier les attaques intra-domaine avant d'envisager des mouvements latéraux inter-domaines.

Topologie du Domaine ESSOS.LOCAL :

- **Contrôleur de Domaine (DC)** : La machine `meereen.essos.local` (Windows Server 2019) héberge les services d'annuaire (AD DS), le DNS et le centre de distribution de clés Kerberos (KDC). Elle est le "joyau de la couronne" à protéger.
- **Serveur Membre** : La machine `braavos.essos.local` (Windows Server 2016) simule un serveur d'application ou de fichiers (IIS, SQL), offrant une surface d'attaque supplémentaire (ex: vulnérabilités web, relais SMB).²
- **Postes Clients** : Des machines Windows 10 ont été intégrées pour simuler l'activité utilisateur et générer du trafic légitime (bruit de fond) indispensable pour tester la capacité de filtrage des SIEM.

1.2. Segmentation Réseau et Virtualisation

L'infrastructure repose sur l'hyperviseur **VirtualBox**, choisi pour sa compatibilité et sa capacité à gérer des réseaux virtuels complexes. Pour respecter le principe de moindre privilège et de cloisonnement, nous avons structuré le réseau en plusieurs zones étanches, reliées par un routeur virtuel (Debian + Suricata) agissant comme analyseur réseau IDS & IPS interne.

2. Réseau simulant un SI d'entreprise (10.203.19.0/25) :

- *Fonction* : Dédié aux serveurs GOAD, aux SIEM & aux serveurs OpenWec.
- *Sécurité* : Ce réseau simule un "VLAN Serveur" classique d'un SI regroupant les serveurs clés et leurs outils de monitoring, respectant les principes de séparation essentiels pour le déploiement d'un réseau sécurisé.

3. **Routeur Debian + Suricata (10.203.19.120/25 ; 10.203.19.254/25) :**

- *Fonction* : Faire la transition entre le réseau interne des serveurs de l'entreprise et le réseau des attaquants.
- *Règles*: Ici, le routeur Suricata va récolter et analyser tout trafic concernant le réseau 10.203.19.0/24, sans faire de prévention : les règles Suricata "classiques" ont été chargées, afin de voir si cette configuration est solide. Il permet ainsi de voir et de tracer les attaques venant d'un réseau plus vulnérables, simulant un autre VLAN du SI, un VLAN utilisateur.

4. **Réseau d'attaque simulant un VLAN Utilisateur (10.203.19.128/25) :**

- *Fonction* : Réseau sur lequel se trouve Exegol : tout trafic de ce réseau passera forcément par le routeur Suricata, et sera donc analysé et logué. Ainsi on simule des attaques ici venant d'un VLAN différent que le VLAN serveur, afin de respecter les schémas classiques de réseaux segmentés par des switch Core ou des Firewalls.
- *Flux* : Tout flux émis par Exegol dans ce réseau passera par le routeur Suricata avant de rejoindre le réseau Serveur.

Cette architecture reproduit bien une architecture de réseau sécurisée respectant les normes de segmentation et d'isolation de cybersécurité.

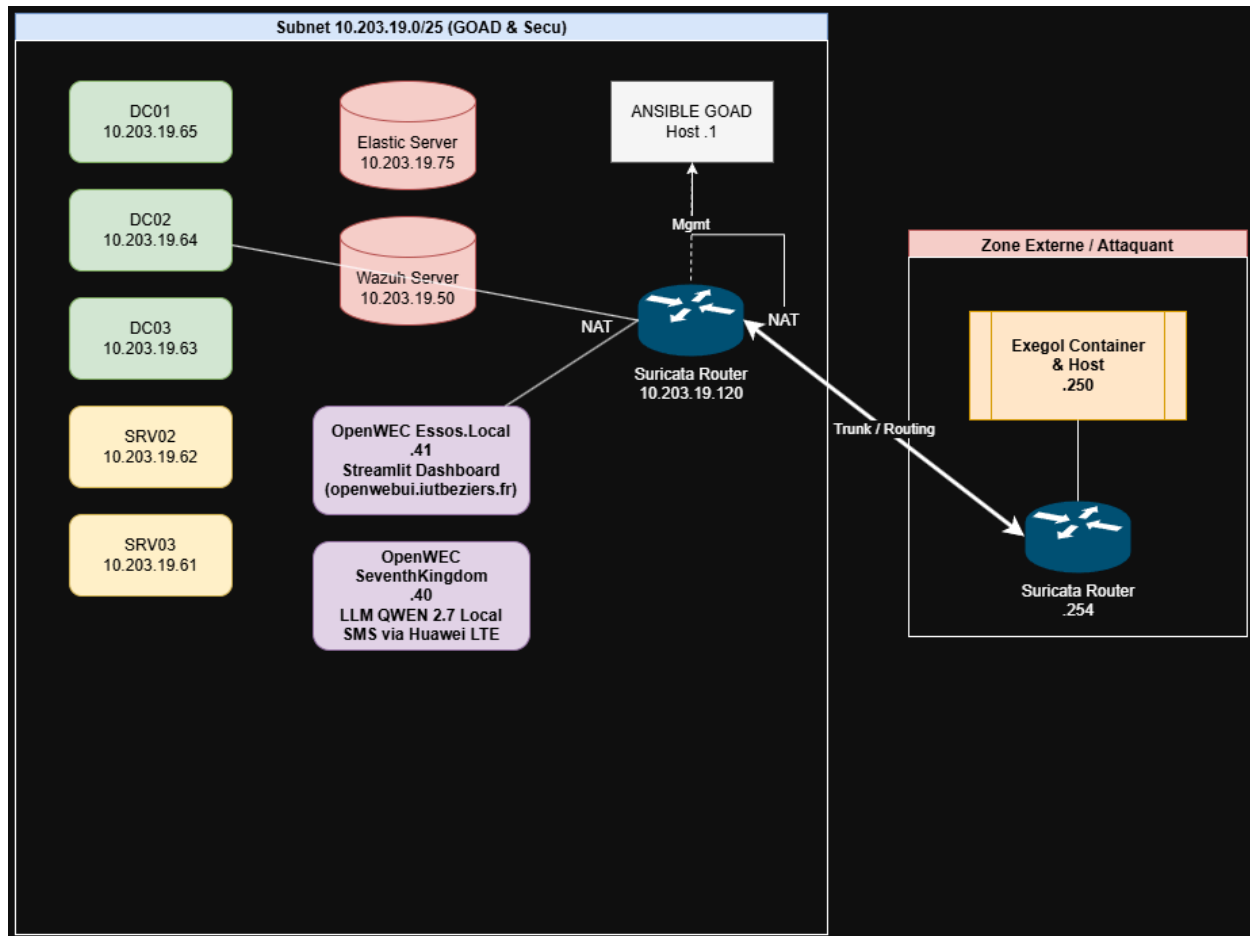


Figure : Schéma de l'infrastructure réseau

2. Services Mis en Place : Déploiement et Présentation

La stratégie de défense repose sur une "stack" technologique complète, couvrant la collecte, l'agrégation, l'indexation, l'analyse et la visualisation des événements de sécurité. Chaque composant a été choisi pour répondre à un besoin spécifique de la chaîne de détection.

2.1. OpenWEC : La Collecte de Logs Nouvelle Génération

Le choix d'**OpenWEC** constitue une décision architecturale majeure. Traditionnellement, la centralisation des journaux Windows via le protocole *Windows Event Forwarding* (WEF) nécessite un serveur Windows dédié agissant comme *Windows Event Collector* (WEC). Cela induit des coûts de licence et une lourdeur de gestion.

OpenWEC, projet open-source, permet d'implémenter un serveur WEC sur une machine Linux. Il reçoit les événements envoyés par les clients Windows via le protocole standard WS-Management (WinRM).

- **Architecture Conteneurisée** : Nous avons déployé OpenWEC sous forme de

conteneurs Docker. Cette approche "Micro-services" permet d'isoler le collecteur, de faciliter sa mise à jour et d'assurer sa résilience. En cas de crash du service, le conteneur redémarre automatiquement sans impacter l'hôte. C'est aussi plus pratique pour rendre l'architecture améliorable à moindre coût : un monitoring Portainer Open-Source est envisageable pour un coût quasi-nul.

- **Mode de Souscription** : Nous avons configuré OpenWEC en mode "**Source-Initiated**" (**Push**). Dans ce mode, ce sont les machines Windows qui contactent le serveur OpenWEC pour envoyer leurs logs. C'est la configuration recommandée pour les environnements de postes de travail (laptops, clients DHCP) dont les adresses IP changent fréquemment, car le serveur n'a pas besoin de connaître l'inventaire IP des clients à l'avance.
- **Ségrégation par Domaine** : Pour garantir une clarté totale des données, nous avons instancié **deux conteneurs OpenWEC distincts**, un pour chaque domaine (bien que l'emphase soit sur ESSOS dans nos attaques). Cela permet d'appliquer des politiques de rétention et de filtrage différenciées.

```
root@avalanche:/opt/openwec# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
f7ca77524c57	openwec	"/entrypoint.sh"	7 days ago	Up 6 hours	0.0.0.0:5985->5985/tcp, [::]:5985->5985/tcp, 5986/tcp	openwec

```
root@avalanche:/opt/openwec#
```

Figure : Conteneur Docker Openwec fonctionnant

2.2. Sysmon : L'Œil Absolu sur le Système

Les journaux d'événements Windows natifs sont riches mais souvent insuffisants pour détecter des techniques d'attaque avancées (comme l'injection de DLL ou le détournement de flux d'exécution). Nous avons donc déployé **Sysmon (System Monitor)**, un pilote de noyau Microsoft, sur l'ensemble des machines Windows.

- **Configuration Avancée** : Sysmon n'est utile que s'il est bien configuré. Nous avons utilisé une configuration XML optimisée (basée sur le standard de l'industrie *SwiftOnSecurity*), adaptée pour filtrer le bruit (événements légitimes) et se concentrer sur les indicateurs de compromission (IOCs) critiques :
 - *Event ID 1 (Process Create)* : Pour détecter l'exécution d'outils d'attaque (Mimikatz, Rubeus).
 - *Event ID 3 (Network Connection)* : Pour corréler les processus avec les connexions réseaux (ex: PowerShell se connectant à une IP externe).
 - *Event ID 10 (Process Access)* : Pour détecter les tentatives de lecture de la mémoire LSASS (dump de crédits).

2.3. Elastic Stack (ELK) : Le Lac de Données

Pour le stockage et l'analyse brute, nous avons déployé un **cluster Elastic** (Elasticsearch, Logstash, Kibana).

- **Rôle** : Elastic agit comme le "Data Lake" de sécurité. Il reçoit les logs bruts normalisés

par OpenWEC et Winlogbeat.

- **Pipeline d'Ingestion** : Logstash a été configuré pour parser les formats JSON envoyés par OpenWEC et les enrichir (Géo-ip, résolution DNS) avant l'indexation.
- **Visualisation** : Kibana offre l'interface d'exploration. Bien que puissant, nous avons constaté qu'Elastic nécessite un effort considérable de configuration pour créer des tableaux de bord de sécurité pertinents ("Engineering cost").



Figure : Tableau de bord du service Elastic déployé

2.4. Wazuh : Le SIEM et XDR Unifié

Wazuh a été sélectionné comme solution SIEM principale. C'est une plateforme Open Source qui combine les fonctionnalités de SIEM (gestion des événements) et de XDR (détection et réponse étendue).

- **Architecture** : Le serveur Wazuh centralise les alertes. Il utilise son propre indexeur (basé sur OpenSearch) et un tableau de bord dédié.
- **Valeur Ajoutée** : Contrairement à un Elastic Stack "nu", Wazuh est livré avec des milliers de règles de détection pré-configurées (mappées sur le framework MITRE ATT&CK). Il intègre également un module de détection de vulnérabilités (scan des logiciels installés) et un module de conformité (PCI-DSS, GDPR), offrant une visibilité immédiate sur la posture de sécurité.

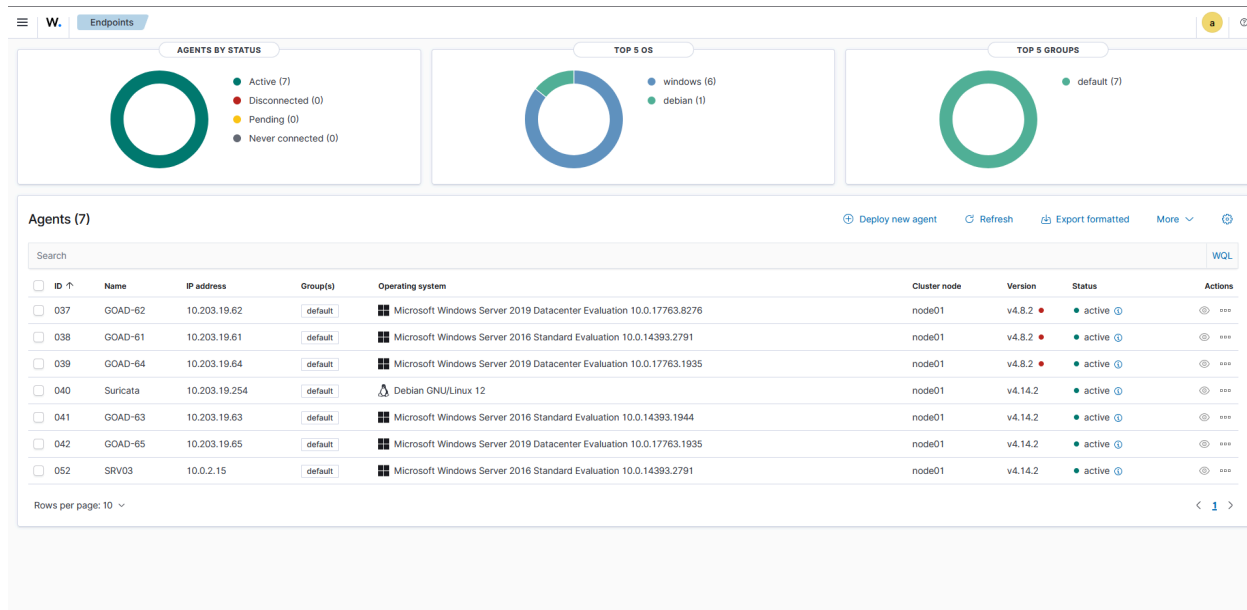
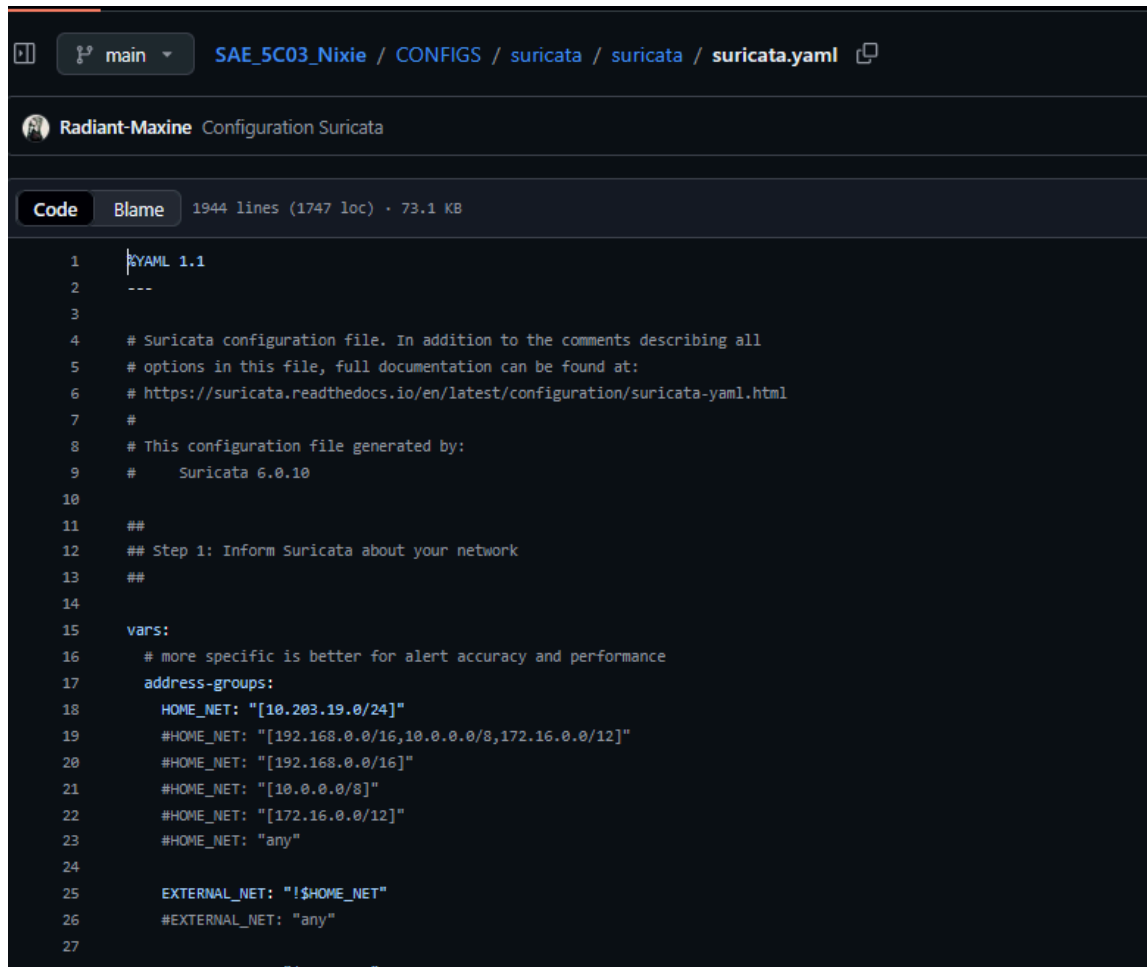


Figure : Interface montrant les différents agents connectés sur Wazuh

2.5. Suricata : Détection d'Intrusion Réseau (NIDS)

Pour compléter la visibilité "hôte" (Host-based), nous avons déployé **Suricata** en tant que NIDS (Network Intrusion Detection System).

- **Positionnement** : Suricata a été placé sur une interface en mode "promiscuous" (port mirroring) au niveau du routeur virtuel, lui permettant d'analyser tout le trafic entrant et sortant du VLAN Corporate.
- **Capacités** : Suricata analyse les flux réseaux à la recherche de signatures d'attaques connues (règles *Emerging Threats*). Il est capable de détecter des scans de ports, des tentatives d'exploitation de services web, ou des communications C2 (Command & Control) que les logs Windows ne verraient pas.



```
1  #YAML 1.1
2  ---
3
4  # Suricata configuration file. In addition to the comments describing all
5  # options in this file, full documentation can be found at:
6  # https://suricata.readthedocs.io/en/latest/configuration/suricata.yaml.html
7  #
8  # This configuration file generated by:
9  #   Suricata 6.0.10
10
11  ##
12  ## Step 1: Inform Suricata about your network
13  ##
14
15  vars:
16    # more specific is better for alert accuracy and performance
17    address-groups:
18      HOME_NET: "[10.203.19.0/24]"
19      #HOME_NET: "[192.168.0.0/16,10.0.0.0/8,172.16.0.0/12]"
20      #HOME_NET: "[192.168.0.0/16]"
21      #HOME_NET: "[10.0.0.0/8]"
22      #HOME_NET: "[172.16.0.0/12]"
23      #HOME_NET: "any"
24
25      EXTERNAL_NET: "!$HOME_NET"
26      #EXTERNAL_NET: "any"
27
28      #UTM_CATEGORIES: "$HOME_NET"
```

Figure : Extrait de la configuration Suricata, disponible sur GitHub

2.6. Innovation : Dashboard Streamlit et Analyse LLM

Pour répondre au besoin de synthèse et d'intelligibilité des données pour des décideurs non-techniques, nous avons développé deux projets **Python** utilisant des **LLMs**.

- **Concept** : Streamlit est un framework Python permettant de créer rapidement des applications de données web.
- **Fonctionnalité** : Les dashboards de l'app Streamlit se connectent aux bases de données de logs OpenWec. Ils extraient les alertes en les triants par horaire, source et autre métriques pour faire des dashboards, et il est possible de les soumettre à un **LLM (Large Language Model)** via API
- **Résultat** : L'application affiche non seulement les graphiques d'attaques, mais aussi une explication en langage naturel générée par l'IA : *"Une activité suspecte a été détectée sur le contrôleur de domaine Meereen, correspondant à une tentative probable de DCSync..."*. C'est une aide à la décision précieuse pour un SOC (Security Operation Center) de niveau 1.

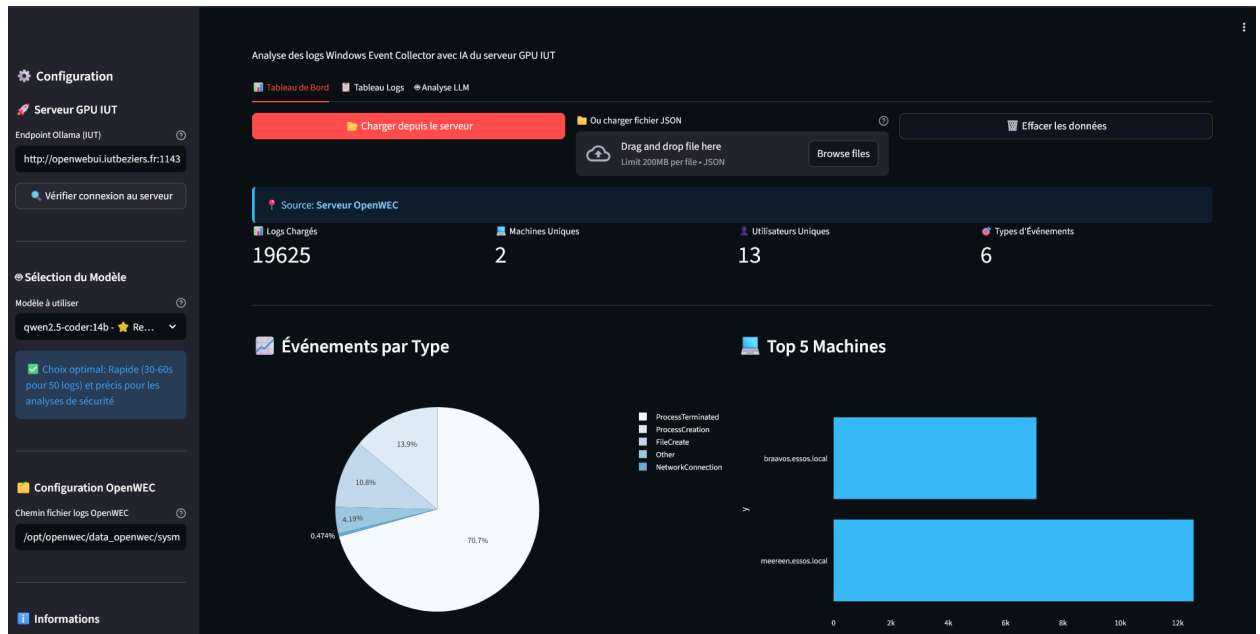


Figure : Capture du dashboard Streamlit

3. Déploiement des Agents et Liaison OpenWEC

L'efficacité d'un SOC dépend de sa couverture (visibility). Pour garantir que 100% des actifs remontent leurs logs, le déploiement manuel est proscrit. Nous avons industrialisé ce processus.

3.1. Automatisation via Ansible

Ansible a été l'outil de choix pour l'orchestration. Sans agent (agentless), il utilise WinRM pour configurer les cibles Windows.

- **Playbook Wazuh** : Nous avons développé un playbook Ansible qui télécharge le MSI de l'agent Wazuh, l'installe silencieusement, injecte la clé d'enregistrement (registration key) auprès du Manager, et démarre le service. Ce script assure que chaque machine est correctement enrôlée dans le SIEM.
- **Playbook Sysmon/Elastic** : De même, des rôles Ansible déploient le binaire Sysmon avec sa configuration XML spécifique et installent l'agent Elastic afin que la communication se fasse automatiquement.

3.2. Configuration de la Collecte OpenWEC par GPO

La liaison entre les clients Windows et le serveur OpenWEC repose sur le mécanisme natif de Windows, piloté par **GPO (Group Policy Object)**.

- **Subscription Manager** : Une GPO "Log Collection" a été créée dans le domaine ESSOS.LOCAL. Elle configure le paramètre *SubscriptionManager* du service WinRM sur tous les postes pour pointer vers l'URL du conteneur OpenWEC :

<http://openwec.security.zone:5985/wsman/SubscriptionManager/WEC>.

- **Sécurité et ACLs** : Pour que cela fonctionne, la GPO configure également les ACLs (Listes de Contrôle d'Accès) des journaux d'événements locaux pour autoriser le compte "Service Réseau" (Network Service) à les lire, car c'est ce compte que WinRM utilise pour l'expédition.
- **Résultat** : Dès qu'une machine rejoint le domaine et applique les GPO (gpupdate), elle commence automatiquement à "pusher" ses logs vers OpenWEC sans aucune installation d'agent tiers, validant l'approche "Agentless" pour la collecte standard.
- **Pourquoi GPO plutôt qu'Ansible ?** : Nous avons choisi pour Openwec de déployer par GPO plutôt que par playbook ansible car les GPOs s'appliquent aussi aux machines clientes qui ne sont pas déployées par Ansible mais quand même reliées au réseau : là où les serveurs déployés par Ansible existent, les machines clientes le sont plus rarement. Ainsi, le déploiement par GPO est l'assurance de couvrir les clients en plus des serveurs.

4. Présentation d'Exegol et Scénarios d'Attaque

Pour tester la robustesse de notre *Blue Team*, nous avons endossé le rôle de *Red Team*. L'outil central de cette phase est **Exegol**.

4.1. L'Environnement Exegol

Exegol est un environnement de hacking professionnel basé sur Docker. Il résout le problème de la "pollution" de la machine attaquante et des dépendances d'outils cassées.²⁴

- **Avantages** : Il fournit une image Docker pré-packagée avec tous les outils modernes d'attaque Active Directory (Impacket, BloodHound, NetExec, Certipy, etc.) pré-installés et configurés.
- **Utilisation** : Depuis notre machine attaquante dans le VLAN 666, nous avons lancé un conteneur Exegol connecté au réseau via VPN ou routage direct, nous permettant d'interagir avec le domaine ESSOS.

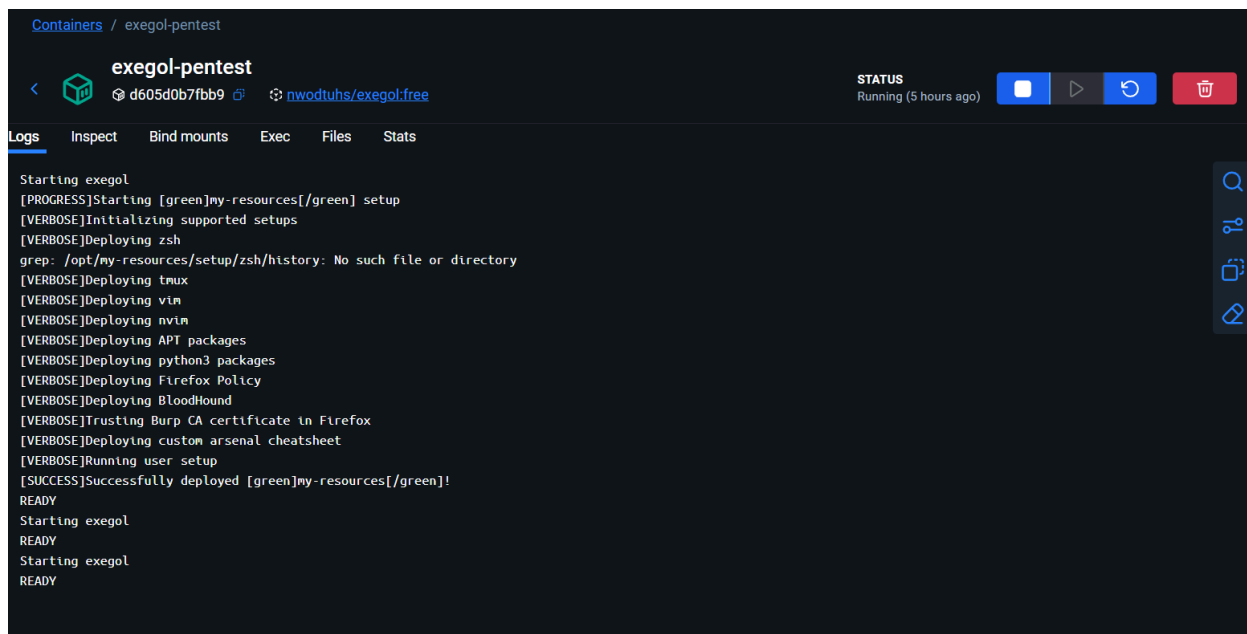


Figure : déploiement d'un conteneur Exegol via Docker Desktop sur Windows.

4.2. Première Attaque : Kerberoasting (Intensité Moyenne)

Le **Kerberoasting** est une attaque emblématique ciblant le protocole Kerberos.

- **Mécanisme** : Tout utilisateur authentifié du domaine peut demander un ticket de service (TGS - Ticket Granting Service) pour n'importe quel service disposant d'un *Service Principal Name* (SPN). Le TGS est chiffré avec le mot de passe du compte de service. Si ce mot de passe est faible, l'attaquant peut craquer le ticket hors ligne pour récupérer le mot de passe en clair.²⁶
- **Exécution sur ESSOS** :
 1. **Reconnaissance** : Utilisation de GetUserSPNs.py (suite Impacket) depuis Exegol pour lister les comptes "roastables" dans ESSOS.LOCAL.
 2. **Extraction** : L'outil demande et récupère le TGS pour un compte de service identifié (ex: svc_sql).
 3. **Craquage** : Tentative de brute-force du TGS avec Hashcat.
- **Empreinte** : Cette attaque est "moyenne" en bruit. Elle génère des événements spécifiques (Event ID 4769) avec un type de chiffrement souvent affaibli (RC4 - 0x17), mais ne nécessite pas de privilèges élevés.

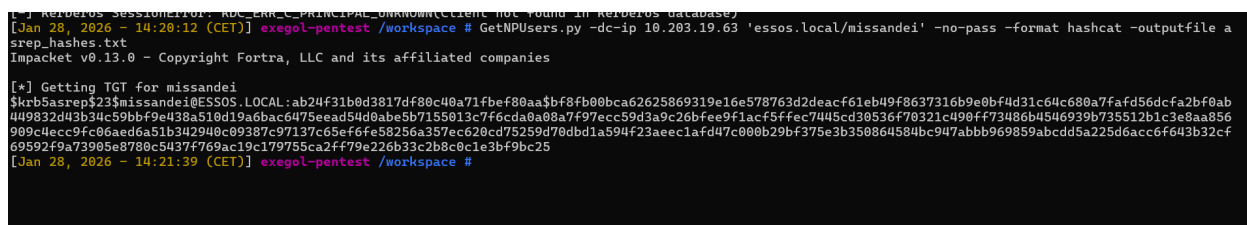


Figure : obtention d'un ticket TGT Kerberos sans mot de passe

4.3. Seconde Attaque : DCSync (Haute Intensité / Critique)

Le **DCSync** est une technique qui simule le comportement d'un Contrôleur de Domaine pour demander à un autre DC de lui répliquer les données sensibles, notamment les hashes de mots de passe.

- **Mécanisme** : Elle exploite le protocole MS-DRSR (Directory Replication Service Remote). Pour l'exécuter, il faut posséder des droits étendus (ex: *DS-Replication-Get-Changes-All*), généralement détenus par les Administrateurs du Domaine.²⁸
- **Exécution** :
 1. Après avoir compromis un compte administrateur (via une autre faille ou un mot de passe faible), nous utilisons `secretsdump.py` (Impacket).
 2. L'outil contacte le DC `meereen.essos.local` et demande la répllication du compte `krbtgt`.
- **Impact** : La récupération du hash NTLM du compte `krbtgt` est catastrophique. Elle permet de forger un **Golden Ticket**, donnant un accès persistant et illimité à l'ensemble du domaine, indétectable par la suite car le ticket est validé par les DCs comme légitime.

```
[Jan 29, 2026 - 13:33:14 [CET]] exegol-pentest /tmp # python3 cve-2020-1472-exploit.py MEEREEN 10.203.19.63
Performing authentication attempts...
=====
=====
=====
=====
=====
=====
Target vulnerable, changing account password to empty string
Result: 0
Exploit complete!
[Jan 29, 2026 - 13:33:32 [CET]] exegol-pentest /tmp #
```

Figure : exécution de l'attaque sur le serveur DC03

5. Traces Récupérées et Analyse Forensique

L'analyse des traces post-attaque permet de valider l'efficacité de notre chaîne de détection.

5.1. Traces dans Elastic Stack (Analyse Brute)

Dans Kibana, nous avons pu retrouver les traces brutes des attaques.

- **Pour Kerberoasting :** En filtrant sur l'Event ID **4769** (Kerberos Service Ticket Requested), nous avons observé une demande de ticket pour le service ciblé. L'analyse du champ TicketEncryptionType montrant 0x17 (RC4) alors que l'environnement utilise AES est un indicateur fort.
- **Pour DCSync :** Les traces sont plus subtiles. L'Event ID **4662** (Operation was performed on an object) est présent, avec le GUID correspondant aux droits de réplication.
- **Constat :** Elastic a capturé toutes les données ("Win" sur la rétention), mais l'identification de l'attaque a nécessité des requêtes manuelles complexes ("Fail" sur l'alerting immédiat sans configuration préalable).

5.2. Traces dans Wazuh (Détection Intelligente)

Wazuh a brillé par sa capacité de corrélation.

- **Alerte DCSync** : Dès l'exécution de secretsdump.py, Wazuh a levé une alerte de niveau critique (Level 12+) intitulée "**Mimikatz usage / DCSync**". Les règles par défaut de Wazuh surveillent spécifiquement les demandes de réplication provenant de comptes non-machines (ne finissant pas par \$), ce qui est la signature exacte de l'attaque.¹³
- **Contextualisation** : L'alerte contenait l'IP source de l'attaquant (la machine Exegol), l'utilisateur compromis utilisé pour l'attaque, et le DC ciblé. Cette clarté est essentielle pour une réponse rapide.

5.3. Traces OpenWEC (Transport)

OpenWEC n'analyse pas, il transporte. Cependant, les logs de diagnostic d'OpenWEC ont montré un pic de volume de logs (Event ID 4624 - Logon, 4688 - Process Create) lors des phases de scan intensif (BloodHound), prouvant que le collecteur tient la charge sans perdre d'événements, même sous stress.³⁰

5.4. Traces Suricata (Visibilité Réseau)

Suricata a fourni le contexte réseau indispensable.

- **Détection Nmap** : Lors de la reconnaissance initiale, Suricata a déclenché des alertes "**ET SCAN Potential Nmap Scan**", indiquant qu'une machine (Exegol) scannait le réseau.
- **Confirmation Kerberoasting** : Suricata a détecté un trafic Kerberos (Port 88) anormalement élevé et des requêtes TGS malformées ou suspectes, corroborant les logs Windows. Cette "double validation" (Réseau + Hôte) réduit drastiquement les faux positifs.¹⁷

6. Comparaison Stratégique des SIEMs

Pour le client final, le choix de la solution de détection est critique. Voici notre analyse comparative basée sur le déploiement réel.

Critère	Wazuh	Elastic Stack (ELK)	OpenWEC
Nature	SIEM & XDR (Sécurité)	Moteur de Recherche & Analytics	Collecteur de Logs (Transport)
Déploiement	Modéré : Scripts d'install tout-en-un, agents unifiés.	Complexe : Nécessite de configurer manuellement les pipelines Logstash, les index patterns, etc.	Simple : Conteneur Docker léger, configuration par fichier TOML.
Efficacité Détection	Excellente : Règles de sécurité prêtes à	Moyenne (par défaut) : Nécessite de créer	Nulle : Ce n'est pas son rôle. Il ne fait que

	l'emploi (HIDS, Malware, Rootkit). Détecte DCSync nativement.	ses propres règles de détection (sauf licence payante Elastic Security).	passer les plats. La détection doit être implémentée par soi-même.
Visualisation	Tableaux de bord orientés Sécurité (Conformité, Menaces).	Tableaux de bord généralistes ultra-flexibles (Data Science).	Aucune interface graphique (CLI uniquement).
Coût (TCO)	Faible (Open Source gratuit).	Moyen à Élevé (Modèle Freemium, fonctionnalités de sécurité avancées payantes).	Très Faible (Open Source, économie de licence Windows).

On peut regarder également via des points techniques plus précis :

Critères d'Évaluation (/10)	Wazuh	Elastic Stack (ELK)	OpenWEC	Suricata
Facilité de Déploiement	8/10	4/10	9/10	7/10
Détection Native (Sécurité)	9/10	5/10	N/A	9/10
Visualisation & Dashboards	8/10	10/10	N/A	N/A
Consommation Ressources	7/10	3/10	10/10	7/10
Facilité de Maintenance	8/10	5/10	9/10	7/10
Flexibilité & Scalabilité	7/10	10/10	6/10	8/10
Rapport Valeur / Temps	10/10	5/10	9/10	9/10
NOTE FINALE (Moyenne)	8.1/10	6.0/10	8.6/10*	7.8/10*

**Note : Les moyennes d'OpenWEC et Suricata sont calculées uniquement sur les critères applicables (N/A exclus), car ce sont des outils spécialisés (Collecte et IDS) qui ne disposent pas d'interface graphique native ou de moteur de corrélation complet.*

Recommandation pour le Client :

- **Quel est le meilleur SIEM? Wazuh.** Il offre le meilleur rapport valeur/temps. Dès l'installation, il détecte des menaces avancées sans configuration complexe. C'est l'outil de choix pour une équipe de sécurité opérationnelle.¹²
 - **Dois-je mettre un IDS? Oui, Suricata.** Il est indispensable. Wazuh ne voit que ce qui se passe *sur* les machines. Si un attaquant utilise un exploit réseau qui ne génère pas de log système immédiat, seul Suricata le verra. La combinaison Wazuh + Suricata offre une couverture totale.
 - **Rôle d'OpenWEC :** Il ne remplace pas le SIEM, mais il est **essentiel** pour l'architecture. Il permet de collecter les logs Windows de manière fiable et gratuite pour les envoyer vers le SIEM (Wazuh ou Elastic) ou simplement pour avoir un layer de collecte centralisé supplémentaire.
-

7. Conclusion

Le projet nous a permis de valider une architecture de cybersécurité moderne, résiliente et adaptée aux menaces actuelles sur les environnements Active Directory.

Nous avons démontré que :

1. **L'Open Source est viable et performant :** L'alliance d'OpenWEC, Wazuh et Suricata rivalise avec des solutions commerciales coûteuses, offrant une détection précise d'attaques complexes comme le DCSync.
2. **L'Automatisation est obligatoire :** Le déploiement via Ansible et GPO garantit que la sécurité est appliquée uniformément, réduisant les "trous dans la raquette" (Shadow IT).
3. **La visibilité doit être totale :** La collecte de logs (Sysmon/OpenWEC) couplée à l'analyse réseau (Suricata) est le seul moyen de contrer des attaquants qui tentent d'effacer leurs traces sur les hôtes.

En conclusion, pour sécuriser le système d'information de votre entreprise, nous recommandons sans réserve le déploiement de la stack **Wazuh + Suricata**, alimentée par une collecte **OpenWEC**. Cette architecture offre une "Défense en Profondeur" robuste, capable de détecter les signaux faibles avant qu'ils ne deviennent des incidents majeurs.

8. Lien avec le Référentiel de Compétences du BUT R&T (Troisième Année)

Ce projet a permis de mobiliser et de valider de nombreuses compétences techniques et organisationnelles du référentiel, réparties entre les unités d'enseignement

"Sécuriser & Surveiller" et "Administrer".

UE : SÉCURISER & SURVEILLER

Composantes Essentielles (CE)

- **{CE.3} Automatiser les tâches** : L'utilisation de playbooks Ansible pour le déploiement des agents (Wazuh, Sysmon) et de GPO pour la configuration d'OpenWEC a permis d'industrialiser la mise en sécurité sans intervention manuelle répétitive.
- **{CE.4} Surveiller le comportement du réseau** : L'intégration de Suricata en mode IDS sur le routeur central a assuré une analyse continue des flux réseaux, permettant la détection de scans (Nmap) et d'activités suspectes (Kerberos).
- **{CE.6} S'adapter, identifier les limites et proposer des actions de remédiation** : Face aux limitations de ressources du laboratoire virtuel, nous avons pivoté d'une architecture OpenWEC classique vers une version conteneurisée sous Docker, garantissant la viabilité du projet.

Situations Professionnelles (SP)

- **[SP.3] Surveillance et analyse du système d'information** : La mise en place d'une stack complète de détection (Elastic, Wazuh, Sysmon) a permis de centraliser les journaux et de visualiser en temps réel l'état de sécurité du domaine Active Directory.
- **[SP.5] Gestion d'un incident de sécurité** : La simulation d'attaques Red Team nous a placés en situation de réaction face à des incidents critiques (DCSync, Kerberoasting), validant nos processus d'alerte et d'analyse forensique.

Apprentissages Critiques (AC)

- **(AC.2) Proposer et mettre en œuvre une architecture sécurisée de système d'information** : Nous avons conçu une architecture de "Défense en Profondeur" segmentant le réseau (VLANs), isolant les outils de gestion et sécurisant la collecte de logs via des protocoles standardisés.
- **(AC.4) Superviser la sécurité du système d'information** : La configuration des règles de corrélation dans Wazuh et la création de tableaux de bord (Kibana, Streamlit) ont fourni une vue d'ensemble supervisée de la posture de sécurité.
- **(AC.5) Appliquer une méthodologie de tests de pénétration** : L'utilisation de l'environnement Exegol a permis de mener méthodiquement des attaques

offensives réalistes pour éprouver la robustesse de la configuration défensive.

UE : ADMINISTRER

Composantes Essentielles (CE)

- **[CE.2] Choisir les solutions et technologies adaptées** : Le choix d'OpenWEC sur Linux plutôt que Windows Server a démontré une capacité à sélectionner des solutions optimisant le rapport coût/performance et l'empreinte ressources.
- **[CE.3] Respecter les principes fondamentaux de la sécurité informatique** : L'architecture respecte le principe de moindre privilège (segmentation réseau, ACLs strictes sur les journaux d'événements) et la traçabilité des actions.
- **[CE.5] Assurer une veille technologique** : L'intégration innovante de l'IA (LLM via Streamlit) pour l'analyse de logs témoigne d'une veille active sur les technologies émergentes applicables à la cybersécurité.

Situations Professionnelles (SP)

- **{SP.1} Conception et administration de l'infrastructure informatique d'une entreprise** : Le projet a nécessité l'administration avancée d'un environnement Active Directory complexe (GOAD), incluant la gestion des DNS, des utilisateurs et des relations d'approbation.
- **{SP.5} Utilisation des technologies de la virtualisation** : L'infrastructure repose intégralement sur l'imbrication de technologies de virtualisation (VirtualBox pour le réseau) et de conteneurisation (Docker pour les services de sécurité).

Apprentissages Critiques (AC)

- **(AC.1) Concevoir un projet de réseau informatique... en intégrant la sécurité** : La topologie réseau a été conçue pour intégrer nativement des sondes de sécurité (port mirroring pour Suricata) sans impacter la disponibilité des services.
- **(AC.2) Réaliser la documentation technique du projet** : Une documentation exhaustive, assistée par IA, a été produite pour détailler les procédures d'installation (Ansible, Docker) et assurer la reproductibilité de l'infrastructure.
- **(AC.5) Gérer le projet et les différentes étapes de sa mise en œuvre** : L'adoption d'une méthodologie Agile avec un suivi par diagramme de Gantt a permis de structurer le travail en équipe et de respecter les délais courts (sprints d'une semaine).

